5.10 Modelo de Objetos del Documento

Una **página web** es un documento HTML que es interpretado por los navegadores en forma gráfica, pero también permiten el acceso al código.

El **Modelo de Objetos del Documento** (DOM) permite ver el mismo documento de otra manera, describiendo el contenido del documento como un conjunto de objetos que un programa JavaScript puede actuar sobre ellos.

La creación del **Document Object Model** es una de las innovaciones que más ha influido en el desarrollo de las páginas web dinámicas y de las aplicaciones web más complejas.

DOM permite a los programadores web **acceder** y **manipular** las páginas XHTML como si fueran documentos XML. De hecho, DOM se diseñó originalmente para manipular de forma sencilla los documentos XML.

A pesar de sus orígenes, DOM se ha convertido en una utilidad disponible para la mayoría de lenguajes de programación (Java, PHP, JavaScript) y cuyas únicas diferencias se encuentran en la forma de implementarlo.

Acorde al W3C el Modelo de Objetos del Documento es una **interfaz de programación de aplicaciones** (API) para documentos validos HTML y documentos XML bien construidos. Define la estructura lógica de los documentos y el modo en que se accede y manipula.

El DOM permite un acceso a la estructura de una página HTML mediante el mapeo de los elementos de esta página en un **árbol de nodos**. Cada elemento se convierte en un nodo y cada porción de texto en un nodo de texto.

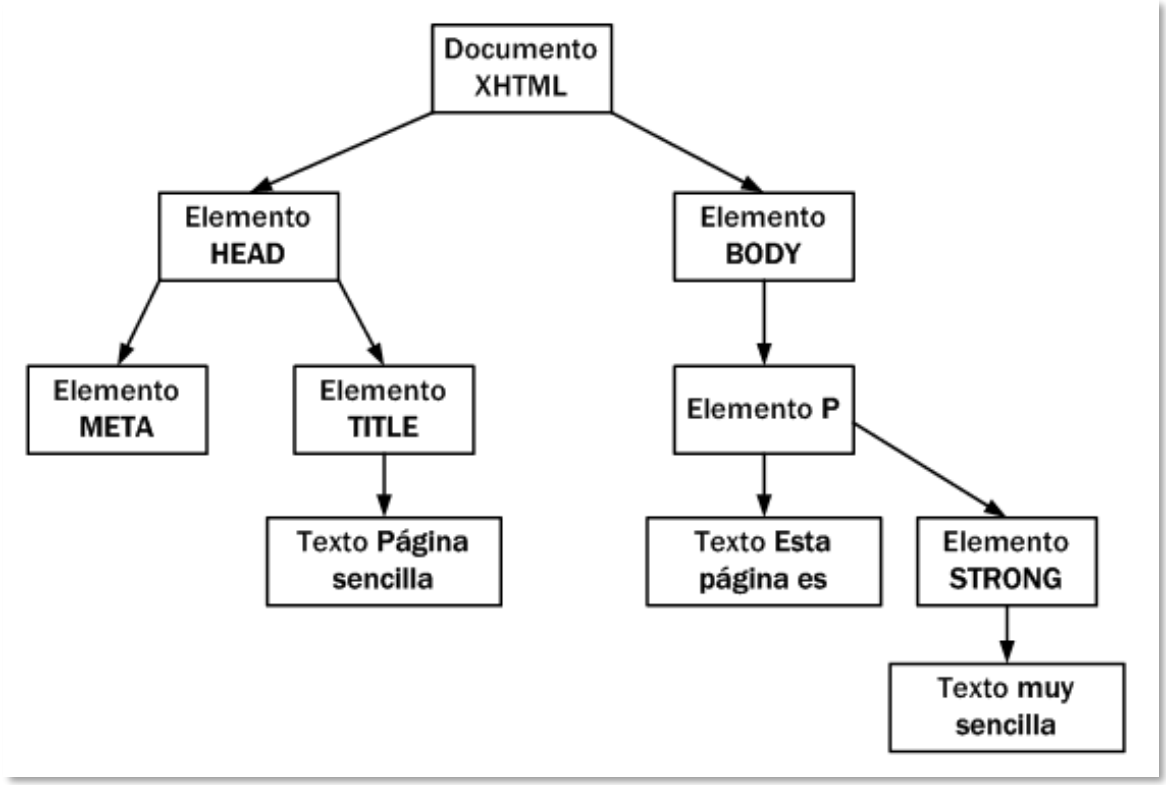
DOM transforma todos los documentos XHTML en un conjunto de elementos llamados **nodos**, que están interconectados y que representan los contenidos de las páginas web y las relaciones entre ellos.

Veamos un ejemplo:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Página sencilla</title>
</head>
<body>
    <p>Esta página es <strong>muy sencilla</strong></p>
</body>
</html>
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

Se transforma en el siguiente árbol de nodos:



En el esquema anterior, cada rectángulo representa un nodo DOM y las flechas indican las relaciones entre nodos. Dentro de cada nodo, se ha incluido su tipo (que se verá más adelante) y su contenido.

La **raíz del árbol** de nodos de cualquier página XHTML siempre es la misma: un nodo de tipo especial denominado "Documento".

A partir de ese nodo raíz, cada etiqueta XHTML se transforma en un nodo de tipo "Elemento". La conversión de etiquetas en nodos se realiza de forma jerárquica. De esta forma, del nodo raíz solamente pueden derivar los nodos HEAD y BODY. A partir de esta derivación inicial, cada etiqueta XHTML se transforma en un nodo que deriva del nodo correspondiente a su "etiqueta padre".

La transformación de las etiquetas XHTML habituales genera dos nodos: el primero es el nodo de tipo "Elemento" (correspondiente a la propia etiqueta XHTML) y el segundo es un nodo de tipo "Texto" que contiene el texto encerrado por esa etiqueta XHTML.

Así, la siguiente etiqueta XHTML:

```
<title>Página sencilla</title>
```

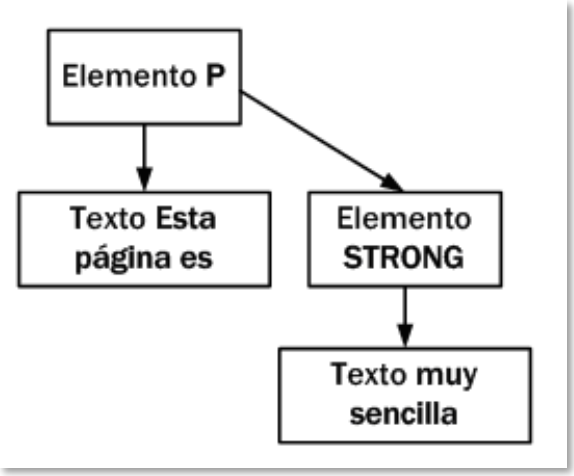
Genera los siguientes dos nodos:

www.grupostudium.com
informacion@grupostudium.com
954 539 952

De la misma forma, la siguiente etiqueta XHTML:

```
<p>Esta página es <strong>muy sencilla</strong></p>
```

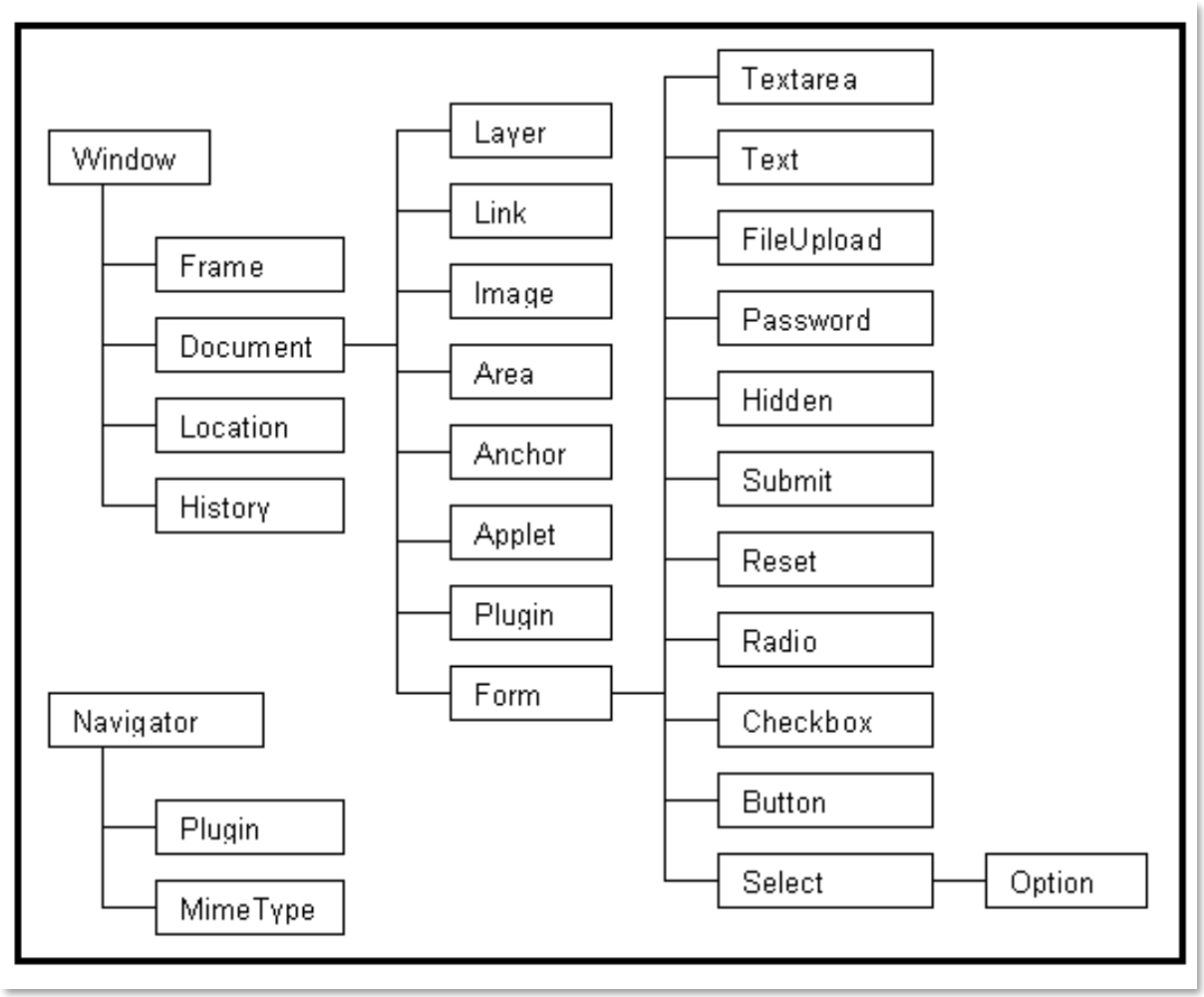
Genera:



Como se puede suponer, las páginas XHTML habituales producen árboles con miles de nodos. Aun así, el proceso de transformación es **rápido y automático**, siendo las funciones proporcionadas por DOM (que se verán más adelante) las únicas que permiten acceder a cualquier nodo de la página de forma sencilla e inmediata.

En la siguiente imagen podemos ver el modelo de objetos común de los navegadores:

www.grupostudium.com
informacion@grupostudium.com
954 539 952



Tipos de nodos

La especificación completa de DOM define 12 tipos de nodos, aunque las páginas XHTML habituales se pueden manipular manejando solamente cuatro o cinco tipos de nodos:

- **Document**, nodo raíz del que derivan todos los demás nodos del árbol.
- **Element**, representa cada una de las etiquetas XHTML. Se trata del único nodo que puede contener atributos y el único del que pueden derivar otros nodos.
- **Attr**, se define un nodo de este tipo para representar cada uno de los atributos de las etiquetas XHTML, es decir, uno por cada par atributo=valor.
- **Text**, nodo que contiene el texto encerrado por una etiqueta XHTML.
- **Comment**, representa los comentarios incluidos en la página XHTML.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Los otros tipos de nodos existentes que no se van a considerar son DocumentType, CDataSection, DocumentFragment, Entity, EntityReference, ProcessingInstruction y Notation.

Acceso a los nodos

Una vez construido automáticamente el árbol completo de nodos DOM, ya es posible utilizar las funciones DOM para **acceder** de forma directa a cualquier nodo del árbol. Como acceder a un nodo del árbol es equivalente a acceder a "un trozo" de la página, una vez construido el árbol, ya es posible manipular de forma sencilla la página: acceder al valor de un elemento, establecer el valor de un elemento, mover un elemento de la página, crear y añadir nuevos elementos, etc.

Las funciones que proporciona DOM para acceder a un nodo a través de sus nodos padre consisten en acceder al nodo raíz de la página y después a sus nodos hijos y a los nodos hijos de esos hijos y así sucesivamente hasta el último nodo de la rama terminada por el nodo buscado. Sin embargo, cuando se quiere acceder a un nodo específico, es mucho más rápido acceder directamente a ese nodo y no llegar hasta él descendiendo a través de todos sus nodos padre.

Por ese motivo, no se van a presentar las funciones necesarias para el acceso jerárquico de nodos y se muestran solamente las que permiten acceder de forma directa a los nodos.

Por último, es importante recordar que el acceso a los nodos, su modificación y su eliminación solamente es posible cuando el árbol DOM ha sido construido completamente, es decir, después de que la página XHTML se cargue por completo. Más adelante se verá cómo asegurar que un código JavaScript solamente se ejecute cuando el navegador ha cargado entera la página XHTML.

getElementsByTagName()

Como sucede con todas las funciones que proporciona DOM, la función *getElementsByTagName()* tiene un nombre muy largo, pero que lo hace autoexplicativo.

La función *getElementsByTagName(nombreEtiqueta)* obtiene todos los elementos de la página XHTML cuya etiqueta sea igual que el parámetro que se le pasa a la función.

El siguiente ejemplo muestra cómo obtener todos los párrafos de una página XHTML:

```
let parrafos = document.getElementsByTagName("p");
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

El valor que se indica delante del nombre de la función (en este caso, *document*) es el nodo a partir del cual se realiza la búsqueda de los elementos. En este caso, como se quieren obtener todos los párrafos de la página, se utiliza el valor *document* como punto de partida de la búsqueda.

El valor que devuelve la función es un array con todos los nodos que cumplen la condición de que su etiqueta coincide con el parámetro proporcionado. El valor devuelto es un array de nodos DOM, no un array de cadenas de texto o un array de objetos normales. Por lo tanto, se debe procesar cada valor del array de la forma que se muestra en las siguientes secciones.

De este modo, se puede obtener el primer párrafo de la página de la siguiente manera:

```
let primerParrafo = parrafos[0];
```

De la misma forma, se podrían recorrer todos los párrafos de la página con el siguiente código:

```
for(let i=0; i<parrafos.length; i++)
{
    let parrafo = parrafos[i];
}
```

La función *getElementsByTagName()* se puede aplicar de forma recursiva sobre cada uno de los nodos devueltos por la función. En el siguiente ejemplo, se obtienen todos los enlaces del primer párrafo de la página:

```
let parrafos = document.getElementsByTagName("p");
let primerParrafo = parrafos[0];
let enlaces = primerParrafo.getElementsByTagName("a");
```

getElementsByName()

La función *getElementsByName()* es similar a la anterior, pero en este caso se buscan los elementos cuyo atributo *name* sea igual al parámetro proporcionado. En el siguiente ejemplo, se obtiene directamente el único párrafo con el nombre indicado:

```
let parrafoEspecial = document.getElementsByName("especial");
```

```
<p name="prueba">...</p>
<p name="especial">...</p>
<p>...</p>
```

Normalmente el atributo *name* es único para los elementos HTML que lo definen, por lo que es un método muy práctico para acceder directamente al nodo deseado. En el caso de los elementos HTML *radiobutton*, el atributo *name* es común a todos los *radiobutton* que están relacionados, por lo que la función devuelve una colección de elementos.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

getElementById()

La función *getElementById()* es la más utilizada cuando se desarrollan aplicaciones web dinámicas. Se trata de la función preferida para acceder directamente a un nodo y poder leer o modificar sus propiedades.

La función *getElementById()* devuelve el elemento XHTML cuyo atributo "id" coincide con el parámetro indicado en la función. Como el atributo "id" debe ser único para cada elemento de una misma página, la función devuelve únicamente el nodo deseado:

```
let cabecera = document.getElementById("cabecera");
```

```
<div id="cabecera">
    <a href="/" id="logo">...</a>
</div>
```

La función *getElementById()* es tan importante y tan utilizada en todas las aplicaciones web, que casi todos los ejemplos y ejercicios que siguen la utilizan constantemente.

Creación y eliminación de nodos

Acceder a los nodos y a sus propiedades (que se verá más adelante) es sólo una parte de las manipulaciones habituales en las páginas. Las otras operaciones habituales son las de crear y eliminar nodos del árbol DOM, es decir, crear y eliminar "trozos" de la página web.

Creación de elementos XHTML simples

Como se ha visto, un elemento XHTML sencillo, como por ejemplo un párrafo, genera dos nodos: el primer nodo es de tipo *Element* y representa la etiqueta <p> y el segundo nodo es de tipo *Text* y representa el contenido textual de la etiqueta <p>.

Por este motivo, crear y añadir a la página un nuevo elemento XHTML sencillo consta de cuatro pasos diferentes:

- Creación de un nodo de tipo *Element* que represente al elemento.
- Creación de un nodo de tipo *Text* que represente el contenido del elemento.
- Añadir el nodo *Text* como nodo hijo del nodo *Element*.
- Añadir el nodo *Element* a la página, en forma de nodo hijo del nodo correspondiente al lugar en el que se quiere insertar el elemento.

De este modo, si se quiere añadir un párrafo simple al final de una página XHTML, es necesario incluir el siguiente código JavaScript:

```
function crear()
```

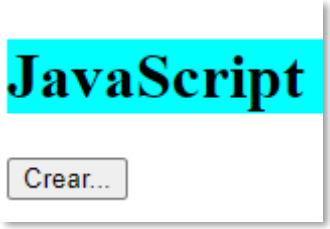

www.grupostudium.com
informacion@grupostudium.com
954 539 952

```
{
    // Crear nodo de tipo Element
    let parrafo = document.createElement("p");
    // Crear nodo de tipo Text
    let contenido = document.createTextNode("Hola Mundo!");
    // Añadir el nodo Text como hijo del nodo Element
    parrafo.appendChild(contenido);
    // Añadir el nodo Element como hijo de la pagina
    document.body.appendChild(parrafo);
}
```

Y el fichero HTML (parte):

```
<body>
    <h1>JavaScript</h1>
    <button onclick="crear();">Crear...</button>
    <script language="javascript" type="text/javascript" src=".\js\index.js">
    </script>
</body>
```

En el navegador partimos de:



Y cada vez que pulsemos el botón, se genera un nodo o elemento nuevo (hijo) y se añade a lo que ya hay:

www.grupostudium.com
informacion@grupostudium.com
954 539 952

El proceso de creación de nuevos nodos puede llegar a ser tedioso, ya que implica la utilización de tres funciones DOM:

- **createElement(etiqueta)**: crea un nodo de tipo *Element* que representa al elemento XHTML cuya etiqueta se pasa como parámetro.
- **createTextNode(contenido)**: crea un nodo de tipo *Text* que almacena el contenido textual de los elementos XHTML.
- **nodoPadre.appendChild(nodoHijo)**: añade un nodo como hijo de otro nodo. Se debe utilizar al menos dos veces con los nodos habituales: en primer lugar, se añade el nodo *Text* como hijo del nodo *Element* y a continuación se añade el nodo *Element* como hijo de algún nodo de la página.

Eliminación de nodos

Afortunadamente, eliminar un nodo del árbol DOM de la página es mucho más sencillo que añadirlo. En este caso, solamente es necesario utilizar la función *removeChild()*. Completemos:

```
let parrafo = document.getElementById("provisional");
parrafo.parentNode.removeChild(parrafo);
```

```
<p id="provisional">...</p>
```

La función *removeChild()* requiere como parámetro el nodo que se va a eliminar. Además, esta función debe ser invocada desde el elemento padre de ese nodo que se quiere eliminar. La forma más segura y rápida de acceder al nodo padre de un elemento es mediante la propiedad nodoHijo.parentNode.

Así, para eliminar un nodo de una página XHTML se invoca a la función *removeChild()* desde el valor *parentNode* del nodo que se quiere eliminar. Cuando se elimina un nodo, también se eliminan automáticamente todos los nodos hijos que tenga, por lo que no es necesario borrar manualmente cada nodo hijo.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Acceso directo a los atributos XHTML

Una vez que se ha accedido a un nodo, el siguiente paso natural consiste en acceder y/o modificar sus atributos y propiedades. Mediante DOM, es posible acceder de forma sencilla a todos los atributos XHTML y todas las propiedades CSS de cualquier elemento de la página.

Los atributos XHTML de los elementos de la página se transforman automáticamente en propiedades de los nodos. Para acceder a su valor, simplemente se indica el nombre del atributo XHTML detrás del nombre del nodo.

El siguiente ejemplo obtiene de forma directa la dirección a la que enlaza el enlace:

```
let enlace = document.getElementById("enlace");
alert(enlace.href); // muestra http://www...com
```

```
<a id="enlace" href="http://www...com">Enlace</a>
```

En el ejemplo anterior, se obtiene el nodo DOM que representa el enlace mediante la función *document.getElementById()*. A continuación, se obtiene el atributo *href* del enlace mediante enlace.href. Para obtener por ejemplo el atributo "id", se utilizaría enlace.id.

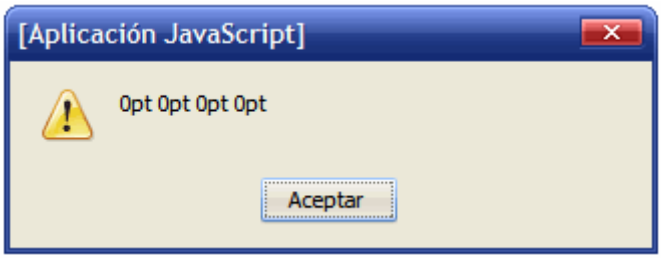
Las propiedades CSS no son tan fáciles de obtener como los atributos XHTML. Para obtener el valor de cualquier propiedad CSS del nodo, se debe utilizar el atributo **style**. El siguiente ejemplo obtiene el valor de la propiedad *margin* de la imagen:

```
let imagen = document.getElementById("imagen");
alert(imagen.style.margin);
```

```
<img id="imagen" style="margin:0; border:0;" src="logo.png" />
```

Aunque el funcionamiento es homogéneo entre distintos navegadores, los resultados no son exactamente iguales, como muestran las siguientes imágenes que son el resultado de ejecutar el código anterior en distintos navegadores:

www.grupostudium.com
informacion@grupostudium.com
954 539 952



Si el nombre de una propiedad CSS es **compuesto**, se accede a su valor modificando ligeramente su nombre:

```
let parrafo = document.getElementById("parrafo");
alert(parrafo.style.fontWeight);  // Muestra "bold"
```

```
<p id="parrafo" style="font-weight: bold;">...</p>
```

La transformación del nombre de las propiedades CSS compuestas consiste en **eliminar** todos los guiones medios (-) y escribir en mayúscula la letra siguiente a cada guion medio. A continuación, se muestran algunos ejemplos:

- ***font-weight*** se transforma en *fontWeight*
- ***line-height*** se transforma en *lineHeight*
- ***border-top-style*** se transforma en *borderTopStyle*
- ***list-style-image*** se transforma en *listStyleImage*

El único atributo XHTML que no tiene el mismo nombre en XHTML y en las propiedades DOM es el atributo **class**. Como la palabra *class* está reservada por JavaScript, no es posible utilizarla para acceder al atributo *class* del elemento XHTML. En su lugar, DOM utiliza el nombre **className** para acceder al atributo *class* de XHTML:

```
let parrafo = document.getElementById("parrafo");
alert(parrafo.class); // Muestra "undefined"
alert(parrafo.className); // Muestra "normal"
```

```
<p id="parrafo" class="normal">...</p>
```

Para finalizar este apartado, vamos a realizar el siguiente ejemplo en el que tendremos un formulario con dos cuadros de texto donde pondremos números y un botón que calculará la suma, mostrando el resultado en otro cuadro de texto.

Fichero HTML:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

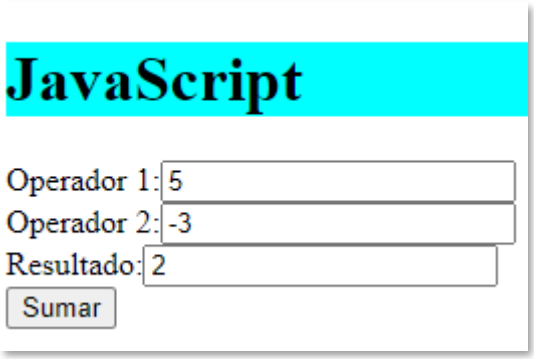
```
    <link rel="stylesheet" href="./css/estilos.css">
    <title>JavaScript</title>
</head>
<body>
    <h1>JavaScript</h1>
    Operador 1:<input type="number" id="operador1" />
    <br />
    Operador 2:<input type="number" id="operador2" />
    <br />
    Resultado:<input type="number" id="resultado" />
    <br />
    <button onclick="sumar();">Sumar</button>
    <script language="javascript" type="text/javascript" src=".\js\index.js">
    </script>
</body>
</html>
```

Fichero js:

```
function sumar()
{
    var operador1 = document.getElementById('operador1').value;
    var operador2 = document.getElementById('operador2').value;
    document.getElementById("resultado").value = parseInt(operador1) + parseInt(operador2);
}
```

La función parseInt() transforma en entero la cadena pasada como parámetro.

En el navegador:



Objeto "window"

Este objeto representa cualquier ventana abierta por el navegador. Actualmente esas ventanas se asimilan a "pestañas" del navegador.

Por ejemplo, el siguiente botón, abrirá una nueva pestaña con la web indicada:

```
<button onclick="window.open('http://www.w3c.es');">Pulsar</button>
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

Es el objeto predeterminado del DOM, de manera que cuando colocamos una instrucción como:

```
alert("Hola, mundo!");
```

Realmente estamos llamando a:

```
window.alert("Hola, mundo!");
```

De esta forma, algunas de las funciones vistas con anterioridad no son tales funciones, sino que se trata en realidad de métodos del objeto *window*.

Otros métodos interesantes de *window* son confirm y prompt, como pudimos ver en el apartado 8.8.

Objeto "location"

Es hijo del objeto *window*. Contiene toda la información referente a la localización u origen del documento XHTML que se muestra en una determinada ventana o pestaña.

Su propiedad más importante es href, y es tanto de lectura como de escritura de manera que podemos mostrar su valor:

```
alert(location.href);
```

Pero también podemos hacer que cambie:

```
location.href="http://www.w3c.es";
```

Objeto "history"

Cuando se navega por la red usando un navegador, se van guardando las páginas que se van visitando en una memoria llamada **historial**. El propio navegador tiene herramientas para avanzar o retroceder en dicho historial o incluso muestra con detalle las visitas realizadas en el día de hoy, ayer, la semana pasada, etc.

Con JavaScript podemos también "movernos" por dicho historial gracias al objeto **history**.

Este objeto tiene una única propiedad llamada length que nos muestra el tamaño de dicho historial, o lo que es lo mismo, cuántas páginas se han visitado. Y también dispone de tres métodos: back(), forward() y go(valor). El primero permite ir hacia atrás en el historial, el segundo permite ir hacia delante y go permite, mediante la inclusión de un número entero como parámetro, ir hacia delante o atrás, tantas posiciones en el historial como indica el número. Así, *history.go(1)* equivaldría a forward(), *history.go(-1)* equivaldría a back(). Cualquier otro valor supondría el salto en dicha cantidad en las posiciones del historial.

www.grupostudium.com
informacion@grupostudium.com
954 539 952

Objeto "screen"

Este objeto nos puede informar sobre características de la pantalla del usuario.

Veamos con un ejemplo las principales propiedades:

```
function mostrar()
{
    console.log(screen.width); // Ancho de la pantalla
    console.log(screen.height); // Alto de la pantalla
    console.log(screen.colorDepth); // Profundidad de colores
}
```

Objeto "document"

Este objeto representa al propio documento que se está mostrando en la ventana del navegador. Se usa para acceder a sus propiedades, así como a los elementos que lo constituyen. Siempre está disponible para cualquier ventana. Sus propiedades y métodos son muy sencillos:

- **title** contiene el título de documento.
- **bgColor** contiene el color de fondo para el documento.
- **fgColor** indica el color para el texto del documento.
- **linkColor** indica el color con el que se resaltarán los hiperenlaces de la página.
- **aLinkColor** especifica el color con que se resaltarán los hiperenlaces activos de la página.
- **vLinkColor** especifica el color con que se resaltarán los hiperenlaces de la página ya visitados y estén en la caché.
- **lastModified** devuelve la fecha de la última modificación del documento actual.

El objeto document posee unas propiedades que son a su vez matrices o colecciones de objetos. Dichas propiedades terminan en -s. Estas colecciones nos permiten acceder a los elementos de la página del tipo que indica su nombre. Veamos algunos ejemplos.

Mostrar todas las anclas o enlaces internos:

```
let anclas = "";
for(i=0; i<document.anchors.length;i++)
{
    anclas += (i+1) + ".- " + document.anchors[i].name + "\n";
}
alert(anclas);
```

Mostrar todos los enlaces:

www.grupostudium.com
informacion@grupostudium.com
954 539 952

```
let enlaces = "";
for(i=0; i<document.links.length;i++)
{
    enlaces += (i+1) + ".- " + document.links[i].href + "\n";
}
alert(enlaces);
```

Mostrar todas las imágenes:

```
let imagenes = "";
for(i=0; i<document.images.length;i++)
{
    imagenes += (i+1) + ".- " + document.images[i].src + "\n";
}
alert(imagenes);
```

Un ejemplo práctico

Para acabar, veremos un ejemplo más elaborado donde pondremos a trabajar algunos de los conceptos que hemos estudiado.

Antes de ver el ejemplo debemos explicar el uso de una función que ejecuta otra función JavaScript al transcurrir cierto tiempo expresado en milisegundos:

```
setTimeout("Función", Tiempo);
```

El código trabajará con una frase que aparecerá letra a letra. Primero el fichero HTML (fragmento):

```
<body onload ="animar();">
    <h1>JavaScript</h1>
    <form>
        <input type="text" name="TextoDinamico" value="" size="120" />
    </form>
    <script language="javascript" type="text/javascript" src=".\js\index.js">
    </script>
</body>
```

Y ahora el fichero JavaScript:

```
let velocidad = 200;
let letras;
let TextoActual = "";
let n = -1;
function animar()
{
    letras = new Array();
    let texto = "Hola que tal!";
    for (i = 0; i < texto.length; i++)
    {
```


www.grupostudium.com
informacion@grupostudium.com
954 539 952

```
        letras[i] = texto.charAt(i);
    }
    mueveLetras();
}
```

```
function mueveLetras()
{
    n++;
    TextoActual += letras[n];
    document.forms[0].TextoDinamico.value = TextoActual;
    if (n == letras.length - 1)
    {
        n = -1;
        TextoActual = "";
    }
    setTimeout("mueveLetras()", velocidad);
}
```

5.11 jQuery

Se trata de una librería de JavaScript que facilita mucho el desarrollo de aplicaciones web y sobre todo mejoran mucho el aspecto gráfico y de eventos, al incorporar multitud de elementos para un diseño más atractivo y rápido, así como todo un elenco de efectos gráficos y de eventos que hacen más dinámicas y agradables a nuestras aplicaciones web.

jQuery

Las definiciones de **jQuery** rondan siempre a que simplifica enormemente el desarrollo de aplicaciones JavaScript y además es fácil de aprender.

Para adentrarnos en el mundo jQuery solamente tenemos que conocer HTML, CSS y JavaScript.

Como su eslogan indica, jQuery es “escribir menos, hacer más”. La idea es recoger funcionalidad de muchas líneas de JavaScript y envolverlas en otra funcionalidad más simple y sencilla.

www.grupostudium.com
informacion@grupostudium.com
954 539 952